JAVA PROGRAMMING PRACTICAL RECORD

Week 5 — Employee Details Using Constructor Overloading

Experiment No.	Week 5 / _____
Date	____ / ____ / _____
Title	Employee class using multiple constructors

1. Aim / Objective

To write and execute a Java program that stores and displays employee details using **constructor overloading**. The program defines multiple constructors in the **Employee** class with different parameter lists so employee objects can be initialized with different amounts of information.

2. System Requirements

Hardware	Standard PC or laptop; Intel Core i3/i5 or equivalent processor; 4 GB RAM or more.
Software	Windows, Linux, or macOS; Java Development Kit (JDK 17 or 21 recommended); a text editor or IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans.

3. Algorithm / Logic Description

1. Start the program.

2. Declare the Employee class with fields for name, ID, designation, salary, and promotion status.

3. Define three overloaded Employee constructors: one accepts name and ID; the second accepts name, ID, designation, and salary; the third accepts all five details.

4. Initialize fields in each constructor. Use `this` to refer to the current object's instance variables.

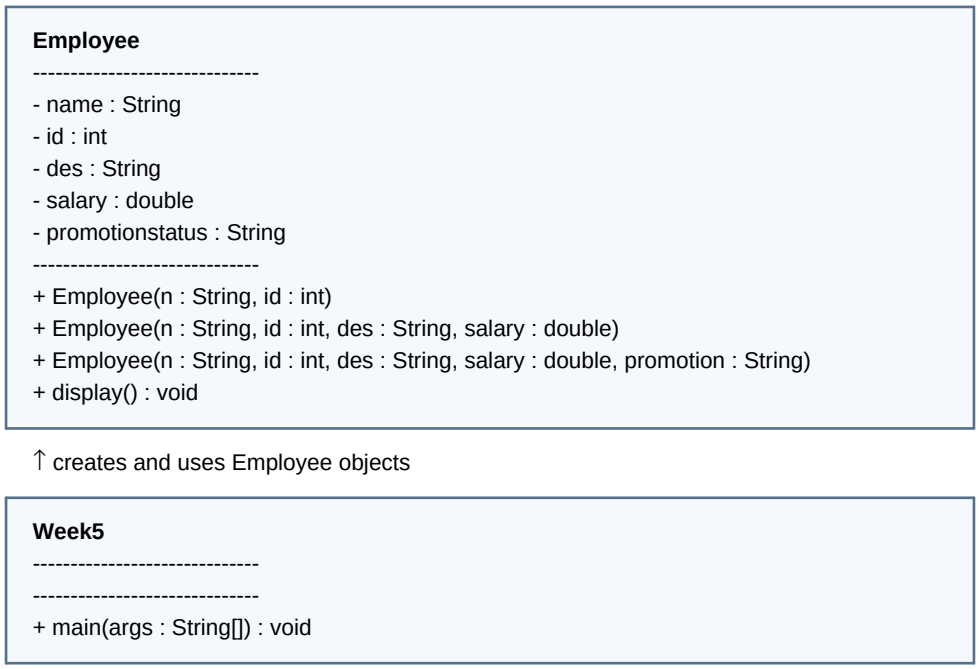
5. Define the `display()` method to print all employee details.

6. In the `Week5` main class, create three Employee objects using the different constructors.

7. Call `display()` for each object, separating the output with blank lines.

8. Stop the program.

4. UML Class Diagram



Note: The relationship is object creation/use, not inheritance. Constructor overloading is shown by the three Employee constructors with different parameter lists.

5. Source Code

Save the following program as **Week5.java**, because `Week5` is the public class name. The code below preserves the program's behavior and removes the extra semicolons after the salary assignments.

```
// Employee class used to get and display employee details
class Employee {
    String name;
    int id;
    String des;
    double salary;
    String promotionstatus;

    // Constructor 1: initializes name and ID only
    Employee(String n, int id) {
        this.name = n;
        this.id = id;
        this.des = "Employee";
        this.salary = 0;
        this.promotionstatus = "unable to promote";
    }

    // Constructor 2: initializes name, ID, designation, and salary
    Employee(String n, int id, String des, double salary) {
        this.name = n;
        this.id = id;
        this.des = des;
        this.salary = salary;
        this.promotionstatus = "pending...";
    }

    // Constructor 3: initializes all employee details
    Employee(String n, int id, String des, double salary,
             String promotion) {
        this.name = n;
        this.id = id;
        this.des = des;
        this.salary = salary;
        this.promotionstatus = promotion;
    }

    // Displays the employee details
    void display() {
        System.out.println("name of the employee:" + name);
        System.out.println("id of the employee:" + id);
        System.out.println("designation of the employee:" + des);
        System.out.println("salary of the employee:" + salary);
        System.out.println("promotion status:" + promotionstatus);
    }
}

// Main class
public class Week5 {
    public static void main(String[] args) {
        System.out.println("employee list");

        Employee e1 = new Employee("raghu", 10000);
        Employee e2 = new Employee("jassu", 1001, "manager", 70000);
        Employee e3 = new Employee(
            "uday", 1002, "ass manager", 65000, "manager"
        );

        e1.display();
        System.out.println(" ");
        e2.display();
        System.out.println(" ");
        e3.display();
    }
}
```

6. Compilation and Execution Commands

Open a terminal in the folder containing Week5.java and run:

```
javac Week5.java
java Week5
```

Important: The filename must match the public class name. Since the public class is `Week5`, use `Week5.java` rather than CarDemo.java.

7. Expected Output Screen

```
employee list
name of the employee:raghu
id of the employee:10000
designation of the employee:Employee
salary of the employee:0.0
promotion status:unable to promote

name of the employee:jassu
id of the employee:1001
designation of the employee:manager
salary of the employee:70000.0
promotion status:pending...

name of the employee:uday
id of the employee:1002
designation of the employee:ass manager
salary of the employee:65000.0
promotion status:manager
```

For a practical record, you may replace this text-based output box with a screenshot of your own terminal after compiling and running the program.

8. Result

The program demonstrates constructor overloading in Java. Three Employee objects are initialized using constructors with different parameter lists, and their details are displayed using the display() method.